To traverse live objects, Ruby provides the method ObjectSpace.each_object, which takes a block and applies that block to every object ObjectSpace knows about. The method takes an optional argument, which is a class or module, and filters the result to only objects that are of that class or module or subclass of it.

For example, to iterate over all objects of type Complex, you'd write the following:

```
a = Complex(1, 2)
b = Complex(99, -100)
ObjectSpace.each_object(Complex) { |x| puts x }
```

produces:

```
99-100i
1+2i
0+1i
```

Where did that extra number, (0+1i), come from? We didn't define it in our program. Well, the Complex class defines a constant for *I*, the square root of -1. Because we are examining all living objects in the system, these turn up as well.

Let's try the same example with different values. This time, they're objects of type Integer:

```
a = 102
b = 95
ObjectSpace.each_object(Integer) { |x| p x }
```

produces:

```
9223372036854775807
```

Neither of the Integers objects we created showed up, although two other integers did. That's because ObjectSpace doesn't know about objects with *immediate* values, which is to say the value is small enough to be basically stored as part of the internal chip instructions rather than a value that needs to be separately stored in memory. In Ruby, Symbol, true, false, and nil are immediate, and Integer and Float values are also immediate if they are close enough to zero (for machine-specific values of "close enough"). (Symbol is weird, actually, because if you do this exercise with Symbol, you'll see a lot of symbols with what seems to be internal-facing names, but new ones you create won't show up.)

For what it's worth, 9223372036854775807 = 2 ** 63 - 1 (we did have to look that up), which suggests it's a memory-specific number being generated internally somewhere, possibly being used as a boundary for integer values.

Looking Inside Objects

Once you've found an interesting object, you may be tempted to find out just what it can do. Unlike static languages, where a variable's type determines its class, and hence the methods it supports, Ruby supports dynamic objects. You really cannot tell exactly what an object can do until you look under its hood. Or under its bonnet, for objects created to the east of the Atlantic. We talk about this in Duck Typing, on page 357.

For instance, we can get a list of all the methods to which an object will respond (these include methods in an object's class and that class's ancestors) with this:

```
r = 1..10 # Create a Range object
list = r.methods
list.length # => 122
```